



Pontificia Universidad Javeriana

Asignatura: Arquitectura de Software

INVESTIGACIÓN & CASO PRÁCTICO

ESTILO ARQUITECTÓNICO DISTRIBUIDO

Arquitectura Orientada a Eventos (EDA)

Análisis de atributos de calidad, evaluación de stack tecnológico y diseño práctico del sistema
FoodFlow

 **Angular**

 **Spring Boot**

 **PostgreSQL**

 **Apache Kafka**

INTEGRANTES DEL GRUPO:

Sara Albarracín • Juan Diego Rojas

Pontificia Universidad Javeriana

Agenda de la Presentación

01



Investigación EDA

Fundamentos, topologías
Broker/Mediator y patrones de
soporte.

02



Stack Tecnológico

Angular, Spring Boot,
PostgreSQL, Kafka y API
Gateway.

03



Análisis Arquitectónico

Matriz de atributos de calidad,
tácticas y mercado laboral.

04



Caso FoodFlow

Dominio de negocio,
entidades y flujo transaccional
E2E.



05. Modelado C4 & Diagramas HLD



06. Guía de Código & Frameworks



07. Conclusiones & Roadmap

Contexto y Motivación del Trabajo

Sistemas Distribuidos Modernos

La arquitectura no solo define componentes aislados, sino la velocidad, flexibilidad y resiliencia con la que interactúan.

Desacoplamiento y Escalabilidad

Las organizaciones requieren comunicación asíncrona para evitar fallas en cascada durante picos masivos de demanda.

Retos de Integración HTTP Tradicional

-  Bloqueos por llamadas HTTP síncronas en cadena
-  Alto acoplamiento entre productores y consumidores

 **Solución EDA:** Intercambio asíncrono de hechos pasados inmutables

Objetivos del Proyecto

OBJETIVO GENERAL

Analizar el estilo EDA, estudiar su relación con un stack tecnológico moderno y aplicarlo en el diseño e implementación del caso práctico FoodFlow.

1. Investigación EDA

Estudiar patrones, topologías y principios fundamentales.

2. Evaluación de Stack

Analizar Angular, Spring Boot, PostgreSQL y Kafka.

3. Atributos de Calidad

Evaluar escalabilidad, resiliencia y consistencia eventual.

4. Caso FoodFlow

Diseñar el dominio, entidades y flujos funcionales.

5. Modelado C4 & HLD

Diagramar niveles Contexto, Contenedor y Despliegue.

6. Guía de Código

Estructurar proyectos reales con frameworks modernos.

SECCIÓN 01 / 06



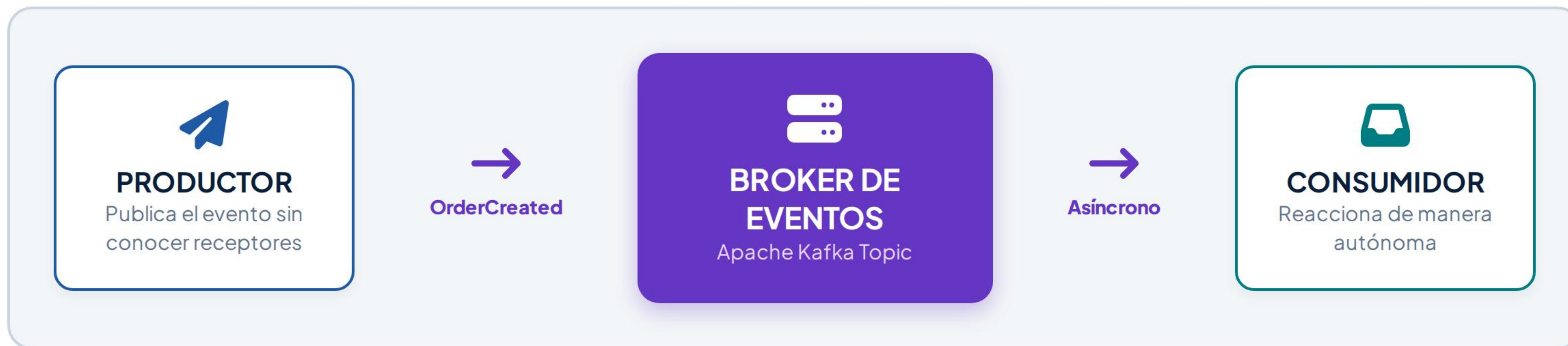
Investigación del Estilo EDA

Conceptos clave, eventos inmutables, topologías de comunicación y patrones de soporte

 Comenzamos entendiendo los fundamentos teóricos del paradigma de eventos.

¿Qué es Event-Driven Architecture?

“ Un Evento representa un HECHO QUE YA OCURRIÓ e inmutable en el dominio.



IDEA CLAVE: Los productores no necesitan conocer a sus consumidores. Los componentes quedan desacoplados totalmente en tiempo y espacio.

Qué ES y qué NO ES EDA

✓ EDA SÍ ES

- ✓ **Estilo de Comunicación:** Notificación de hechos pasados.
- ✓ **Desacoplamiento Total:** El emisor ignora los receptores.
- ✓ **Arquitectura Reactiva:** Base para alta escalabilidad.
- ✓ **Evolución Independiente:** Nuevos consumidores dinámicos.

✗ EDA NO ES

- ✗ **Solo "Usar Colas":** Exige diseño de dominio.
- ✗ **Microservicios Obligatorios:** Aplica en monolitos modulares.
- ✗ **Bala de Plata:** Innecesario en sistemas CRUD simples.
- ✗ **Cero Configuración:** Exige observabilidad y gobierno.

Topologías y Características Principales

Mediator Topology



Orquestación centralizada mediante un mediador que coordina los pasos.

A → Mediator → [B | C | D]

Broker Topology



Coreografía descentralizada. Eventos publicados directamente a canales.

A → Broker → Consumers

Event Streaming



Streams continuos persistentes con capacidad de replay y log distribuido.

Producer → Log → Replay



Desacoplamiento Temporal



Escalabilidad Horizontal



Consistencia Eventual



Reacción Inmediata

Ventajas y Desventajas de EDA

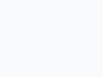
Beneficios Arquitectónicos

- **Alto desacoplamiento:** Autonomía total de tecnología y despliegue.
- **Absorción de picos:** El broker amortigua ráfagas intensas de tráfico.
- **Resiliencia distribuida:** Fallos en consumidores no detienen emisores.
- **Extensibilidad:** Nuevos consumidores sin tocar código existente.

Costos Operativos

- **Depuración compleja:** Requiere trazabilidad distribuida.
- **Consistencia eventual:** La base de datos no es síncrona globalmente.
- **Mensajes duplicados:** Exige idempotencia en todos los consumidores.
- **Curva de aprendizaje:** Monitoreo de lag, particionamiento y brokers.

Problemas Comunes y Patrones Asociados

 Problema Frecuente	 Patrón Asociado	Solución Arquitectónica
 Eventos Duplicados	 Idempotent Consumer	Verifica UUID de evento antes de procesar.
 Inconsistencia BD - Broker	 Transactional Outbox	Escribe evento en BD local y Debezium lo publica.
 Errores de Procesamiento	 Dead Letter Queue (DLQ)	Aísla mensajes fallidos para reprocesamiento.
 Transacciones Prolongadas	 Saga Pattern	Coordinación distribuida compensatoria.



IDEA CLAVE

EDA no funciona de manera aislada; requiere patrones complementarios de soporte para garantizar robustez en producción.

Cuándo Usar y Cuándo Evitar EDA

✓ USAR EDA CUANDO:

- ✓ Existen múltiples consumidores de una misma acción.
- ✓ Se requiere desacoplamiento temporal de procesos.
- ✓ Hay cargas variables y picos masivos de demanda.
- ✓ El dominio de negocio es naturalmente reactivo.

✗ EVITAR EDA CUANDO:

- ✗ El sistema es un CRUD pequeño con baja carga.
- ✗ Se exige consistencia ACID inmediata global.
- ✗ El equipo no posee madurez en monitoreo distribuido.
- ✗ El costo de infraestructura supera el beneficio.

SECCIÓN 02 / 06



Investigación del Stack Tecnológico

Evaluación técnica de Angular, Spring Boot, PostgreSQL, Apache Kafka y
API Gateway

🔗 Ahora que entendemos EDA, necesitamos tecnologías capaces de materializarlo.

Stack Tecnológico Seleccionado



Angular

Frontend Reactivo +
TypeScript



Spring Boot

Backend Java
Microservicios



PostgreSQL

Persistencia Relacional
Local



Apache Kafka

Event Streaming
Backbone



API Gateway

Borde Síncrono
REST/JSON

"Cada tecnología cumple una responsabilidad específica dentro de la arquitectura global."

Angular + TypeScript

Framework robusto para aplicaciones empresariales escalables con arquitectura basada en componentes reactivos.

- ✓ **TypeScript:** Tipado estático y mantenibilidad en proyectos complejos.
- ✓ **RxJS Integrado:** Manejo nativo de Streams de eventos asíncronos en UI.
- ✓ **Inyección de Dependencias:** Modularidad y facilidad de testing.



Esquema de Integración UI

Angular Components — RxJS — HttpClient —▶ API Gateway

BACKEND LAYER

Spring Boot + Java

Framework estándar empresarial para construcción rápida de microservicios productivos desacoplados.



Spring REST

Endpoints JSON y Problem Details.



Spring Kafka

Publicación y consumo simplificado.



Spring Data JPA

Persistencia relacional mapeada.



Bean Validation

Validación declarativa de datos.

PostgreSQL: Cada Servicio Controla su Propio Dato

Motor relacional de alta confiabilidad que garantiza consistencia ACID local dentro de las fronteras de cada microservicio.

PRINCIPIO FUNDAMENTAL EN EDA

La base de datos NO actúa como medio de integración entre servicios. Cada microservicio es dueño exclusivo de su esquema SQL.



Aislamiento de Persistencia

- Order Service —> order_db
- Payment Service —> payment_db
- Notification Service —> notification_db

Apache Kafka: El Backbone Asíncrono

Plataforma de event streaming distribuida que actúa como la columna vertebral asíncrona de todo el sistema.



Topics & Particiones

Ordenamiento garantizado por clave de mensaje y escalabilidad.



Consumer Groups

Lectura paralelizada independiente para múltiples servicios.



Persistencia & Replay

Log distribuido durable en disco para reproducir eventos.

BORDER LAYER

REST / HTTPS + API Gateway

Define el perímetro síncrono del sistema mediante REST sobre HTTPS con esquemas JSON tipados.

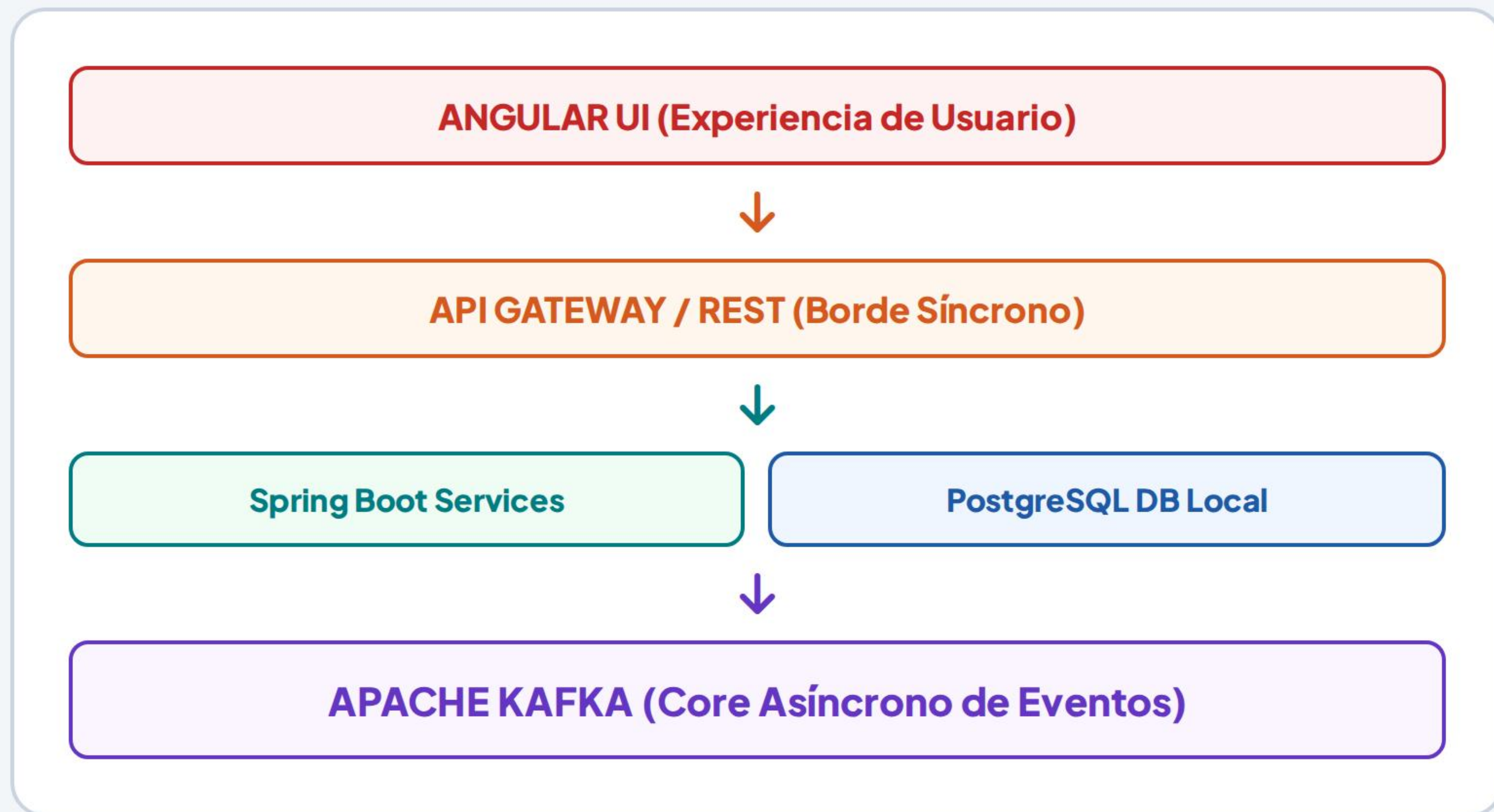
Rol del API Gateway

Centraliza enrutamiento, seguridad, rate limiting y respuestas de error estandarizadas sin exponer los servicios internamente.

Estándares Implementados

- **OpenAPI 3.0:** Contrato formal de API REST.
- **Problem Details (RFC 7807):** Estándar de errores HTTP.

Relación Arquitectónica: REST en el Borde, Kafka en el Core



BORDE SÍNCRONO

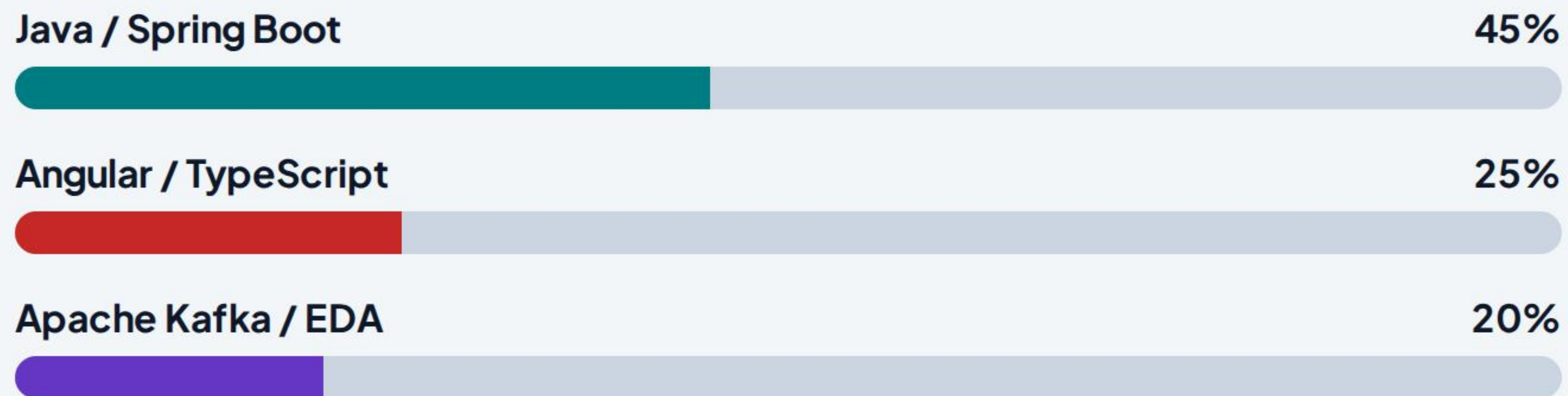
REST sobre HTTPS canaliza peticiones directas del usuario.

CORE ASÍNCRONO

Kafka coordina la ejecución distribuida desacoplada.

Ofertas Publicadas en el Mercado

Distribución de Demanda Laboral Observada



CONCLUSIÓN DE MERCADO

El stack analizado presenta altísima vigencia y demanda real en vacantes empresariales de backend e integración.

SECCIÓN 03 / 06



Análisis Arquitectónico

Matriz de atributos de calidad, principios, tácticas y compensaciones del estilo

🔗 ¿Qué aporta y qué sacrifica el estilo EDA frente a otras alternativas?

Matriz de Atributos de Calidad en EDA

Atributo de Calidad	Soporte EDA	Mecanismo / Táctica Asociada
Escalabilidad	ALTO	Particionamiento Kafka y escalado horizontal.
Disponibilidad	ALTO	Aislamiento de fallos mediante colas asíncronas.
Modificabilidad	ALTO	Inclusión de nuevos consumidores sin alterar el core.
Simplicidad	LIMITADO	Mayor complejidad operativa y trazabilidad.

TRADE-OFF PRINCIPAL

EDA mejora drásticamente el desacoplamiento y la escalabilidad a costa de incrementar la complejidad operativa y aceptar consistencia eventual.

Principios, Tácticas y Patrones



Principios

- Separación de Responsabilidades
- Bajo Acoplamiento
- Alta Cohesión por Dominio
- Autonomía por Servicio



Tácticas

- Validación en Frontera REST
- Reintentos Controlados (DLQ)
- Idempotencia Obligatoria
- Trazabilidad Distribuida



Patrones

- API Gateway
- Database per Service
- Idempotent Consumer
- Transactional Outbox

Rangos Salariales Observados

DESARROLLADOR MID

\$1,800 – \$3,000

USD / Mes Promedio

SENIOR BACKEND KAFKA

\$3,500 – \$5,500

USD / Mes Promedio

ARQUITECTO DE SOFTWARE

\$6,000+

USD / Mes Promedio

SECCIÓN 04 / 06



Caso Práctico: FoodFlow

Diseño de dominio, entidades, flujo funcional E2E y decisiones de arquitectura

 Pasamos del análisis general a una aplicación concreta del estilo.

CASO PRÁCTICO

Sistema de Pedidos FoodFlow

FLUJO CORE EN 10 SEGUNDOS



Pedido, Pago y Notificación colaboran sin llamadas REST directas entre servicios.

Alcance del Prototipo

- ✓ **3 Entidades & Servicios:** Pedido, Pago, Notificación.
- ✓ **3 Bases PostgreSQL:** Aislamiento total SQL.
- ✓ **1 Broker Kafka:** Coordinación asíncrona.
- ✓ **1 Flujo End-to-End:** Desde UI Angular hasta confirmación.

Entidades Principales de FoodFlow



Pedido

- id: UUID
- clientId: String
- montoTotal: Decimal
- estado: Enum
- idempotencyKey: String



Pago

- id: UUID
- orderId: UUID
- monto: Decimal
- estadoPago: Enum
- transaccionId: String



Notificación

- id: UUID
- orderId: UUID
- canal: Enum
- destinatario: String
- estadoEnvio: Enum

Flujo Funcional End-to-End

- Paso 1:** Usuario registra pedido en UI Angular e invoca POST /orders.
- Paso 2:** Order Service valida, persiste pedido CREADO y publica evento **OrderCreated** a Kafka.
- Paso 3:** Payment Service consume OrderCreated y ejecuta simulación transaccional.
- Paso 4:** Payment Service publica evento **PaymentApproved / PaymentRejected** a Kafka.
- Paso 5:** Notification Service consume resultado y despacha confirmación simulada.

Decisiones y Riesgo Aceptado de Escritura Dual

✓ Decisiones Clave

- **Database per Service:** Garantiza autonomía total sin acoplamiento SQL.
- **Idempotency-Key HTTP:** Evita registros duplicados ante reintentos.
- **Event Keying por OrderId:** Orden garantizado en Kafka.

⚠ Riesgo: Escritura Dual sin Outbox

Order Service → 1. COMMIT DB ✓ → 2. PUBLISH Kafka (⚠ Fallo potencial)

Prototipo: Se acepta el riesgo de inconsistencia eventual ante falla de red para acotar alcance.

Evolución Futura: Patrón Transactional Outbox + Debezium.

SECCIÓN 05 / 06



Modelado C4 & Diagramas HLD

Diagramas de alto nivel, niveles de abstracción C4, dinámico, despliegue y modelo de datos

 La arquitectura conceptual ahora debe hacerse visible en diagramas formalizados.

Diagrama de Alto Nivel (HLD)



Insertar Diagrama HLD aquí

Visión macro de la arquitectura del ecosistema FoodFlow

QUÉ OBSERVAR

- Separación clara de fronteras síncronas y asíncronas.
- Ubicación central de Apache Kafka.
- Persistencia distribuida aislada por servicio.

C4 System Landscape



Insertar C4 System Landscape aquí

QUÉ OBSERVAR

- Ubicación de FoodFlow frente a usuarios finales.
- Actores de restaurante y clientes.
- Integración con proveedores externos de correo.

C4 Context Diagram (Nivel 1)



Insertar C4 Context Diagram (Nivel 1)

QUÉ OBSERVAR

- Sistema FoodFlow como caja negra central.
- Interacciones del cliente vía cliente Web.
- Entradas y salidas del límite organizacional.

C4 Container Diagram (Nivel 2)

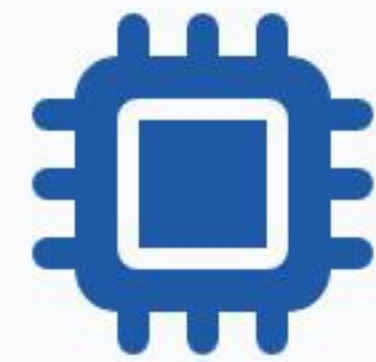


Insertar C4 Container Diagram (Nivel 2)

QUÉ OBSERVAR

- Angular Single Page App
- Spring Cloud API Gateway
- Order, Payment & Notification
- Broker Apache Kafka
- **Las bases PostgreSQL se conectan a su servicio propietario, NUNCA a Kafka.**

C4 Component Diagram (Order Service)



Insertar C4 Component Diagram aquí

QUÉ OBSERVAR

- OrderController (REST)
- OrderApplicationService
- OrderValidator
- OrderRepository (JPA)
- KafkaEventPublisher

Diagrama de Clases UML / Código



Insertar Diagrama UML de Código

QUÉ OBSERVAR

- Mapeo directo de componentes C3 a clases Java.
- Interfaces DTO para eventos Kafka.
- Anotaciones Spring Framework.

C4 Dynamic Diagram (Runtime)



Insertar C4 Dynamic Diagram aquí

QUÉ OBSERVAR

1. POST /orders HTTP
2. OrderCreated en Kafka
3. Consumer Payment
4. PaymentApproved en Kafka
5. NotificationSent

C4 Deployment Diagram



Insertar C4 Deployment Diagram aquí

QUÉ OBSERVAR

- Client Browser Node
- Docker Engine Host
- Spring Boot Containers
- Cluster Apache Kafka
- Contenedores PostgreSQL

Modelo de Datos Relacional (ERD)



Insertar Modelo de Datos ERD aquí

QUÉ OBSERVAR

Esquemas independientes por servicio con claves primarias UUID e índices SQL aislados.

SECCIÓN 06 / 06



Implementación & Frameworks

Bootstrap inicial, estructura de carpetas, snippets de código y trazabilidad arquitectónica

[🔗](#) Del modelo arquitectónico pasamos a estructuras reales de código.

Inicialización del Stack

Frontend (Angular CLI)

```
$ ng new foodflow-web --  
routing --style=scss
```

Backend (Spring Initializr)

- ✓ Spring Web
- ✓ Spring Data JPA
- ✓ Spring for Apache Kafka
- ✓ Validation & PostgreSQL Driver

Infraestructura Docker

```
$ docker compose up -d
```

Kafka+Zookeeper+3 DBs

Trazabilidad: C4 Component ↔ Estructura de Paquetes

foodflow-web (Angular)

```
|— src/app/  
| |— core/services/ (HTTP / RxJS)  
| |— features/orders/ (Components)  
| |— shared/models/ (TypeScript Interfaces)
```

order-service (Spring Boot)

```
|— controller/ (OrderController - REST)  
|— service/ (OrderApplicationService)  
|— repository/ (OrderRepository - JPA)  
|— event/ (@KafkaListener & KafkaTemplate)
```

Mecanismos de Robustez Implementados

 Validaciones Bean
Validation

 Idempotency-Key HTTP

 Problem Details RFC
7807

 Persistencia Real SQL

Recordatorio de Riesgo:

Sin Outbox, un fallo crítico entre DB commit y Kafka send requeriría reconciliación manual.

Conclusiones y Roadmap Futuro

Conclusiones Clave

- EDA otorga alto desacoplamiento y escalabilidad.
- El stack analizado es coherente y vigente en la industria.
- FoodFlow demuestra la viabilidad técnica E2E.

Evolución Futura

- Incorporar Transactional Outbox (Debezium).
- Implementar Dead Letter Queue (DLQ) avanzada.
- Trazabilidad distribuida con OpenTelemetry.



¿Preguntas?

¡Muchas gracias por su atención!

Sara Albarracín • Juan Diego Rojas

Pontificia Universidad Javeriana — Arquitectura de Software